KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Memory management
Lab 4
Geavanceerde computerarchitectuur

Wouter Fouquet
Florian Vandenbogaerde
Master elektronica ICT
Academiejaar 2024-2025

CONTENTS

Exercise **1**
0.1 Assignment 1
0.2 Pointers - Part 1 1
0.3 Pointers - Part 2 1

1 Methodology **2**
1.1 Part 1: Coalesced memory acces 2
1.1.1 Measurements 2
1.1.2 Images 4
1.1.3 Guiding questions - Part 1 6
1.2 Part 2: Global, shared and constant memory 7
1.2.1 Migration time 7
1.2.2 Different memory types 7
1.2.3 Guiding questions - Part 2 8

EXERCISE

0.1 Assignment

Part 1 - Coalesced memory access

- Design a kernel that inverts the red color component in an image.
- Evaluate execution time for coalesced and un-coalesced memory access.

Part 2 - Glabal, shared, and constant memory

- Design a kernel that multiplies two matrices.
- Assess the processing time for a kernel that utilizes global-only, global and shared, and global and constant memory.

0.2 Pointers - Part 1

- Load up an image using last time's code or make a 1D array of your liking.
- Use one array which sorts data as RGBRGB...RGB and one as RR...RGG...GBB...B.
- Design two kernels, each for processing one of the above arrays.
- Calculate the inverse of the red color component, R = 255 - R.
- Time the kernels for different input and/ or grid sizes.

0.3 Pointers - Part 2

- Define an array that represents a matrix - in 1D or 2D from (start off with small square matrices and simple numbers for easier debugging).
- In this way define/ allocate memory for three matrices, which will serve for C = AB.
- Make a kernel that uses only global memory to compute C - verify the results.
- Make a second and third matrix multiplication kernel, whick use shared and constant memory, respectively, in addition to global memory.
- Time the kernels for different input and/ or grid sizes.

1.1 Part 1: Coalesced memory acces

In this section, we analyze the performance of a GPU-accelerated program that inverts red pixels using coalesced and uncoalesced memory. We have implemented and timed the execution for both memory access patterns, combined with two data layouts: planar and interleaved bitstreams. To gain deeper insights, we measure the execution time of the kernels across different input sizes and grid configurations. Specifically, we perform measurements on three different image resolutions: the original image (960x1280), a lower-resolution version (480x640), and a higher-resolution version (1920x2560). By comparing these methods across varying input sizes, we aim to determine the most efficient approach for red pixel inversion in terms of execution time. The code snippet can be found in attachment 1.1.

1.1.1 Measurements

The results are visible in tables 1.1, 1.2 and 1.3.

Threads per block	**Blocks per grid**	**Interleaved [ms]**		**Planar [ms]**	
		Coalesced	**Uncoalesced**	**Coalesced**	**Uncoalesced**
1	1228800	7,52	7,45	6,15	6,15
2	614400	3,64	3,76	3,17	3,17
4	307200	1,85	1,90	1,61	1,61
8	153600	0,97	1,00	0,86	0,85
16	76800	0,52	0,55	0,49	0,47
32	38400	0,28	0,29	0,31	0,31
64	19200	0,16	0,17	0,18	0,18
128	9600	0,17	0,16	0,18	0,17
256	4800	0,15	0,17	0,16	0,18
512	2400	0,16	0,15	0,18	0,18

Tabel 1.1: Measurements orginal picture (960x1280)

Threads per block	Blocks per grid	Interleaved [ms]		Planar [ms]	
		Coalesced	Uncoalesced	Coalesced	Uncoalesced
1	307200	1,85	1,91	1,60	1,60
2	153600	0,95	0,98	0,84	0,83
4	76800	0,51	0,52	0,46	0,45
8	38400	0,27	0,28	0,25	0,24
16	19200	0,16	0,16	0,16	0,17
32	9600	0,10	0,11	0,12	0,10
64	4800	0,08	0,08	0,08	0,07
128	2400	0,07	0,08	0,08	0,07
256	1200	0,07	0,08	0,07	0,07
512	600	0,07	0,07	0,08	0,07
1024	300	0,07	0,08	0,07	0,08

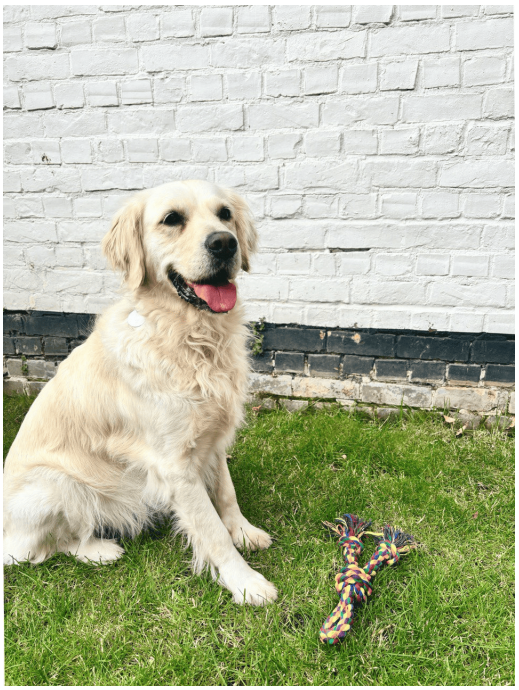
Tabel 1.2: Measurements downscaled picture (480x640)

Threads per block	Blocks per grid	Interleaved [ms]		Planar [ms]	
		Coalesced	Uncoalesced	Coalesced	Uncoalesced
1	4915200	28,72	29,25	24,32	24,33
2	2457600	14,32	14,72	12,37	12,37
4	1228800	7,16	7,36	6,24	6,24
8	614400	3,65	3,73	3,25	3,25
16	307200	1,91	1,96	1,77	1,76
32	153600	1,01	1,04	1,05	1,06
64	76800	0,55	0,56	0,57	0,58
128	38400	0,50	0,51	0,57	0,57
256	19200	0,49	0,52	0,58	0,56
512	9600	0,50	0,53	0,58	0,57
1024	4800	0,53	0,56	0,59	0,59

Tabel 1.3: Measurements upscaled picture (1920x2560)

The measurements show that coalesced memory access consistently outperforms uncoalesced access, particularly for smaller thread block sizes, where inefficient global memory transactions cause higher latency. Larger input sizes naturally lead to longer execution times, but the relative performance trends remain consistent, highlighting the importance of memory access patterns regardless of resolution. The planar data layout is generally more efficient than the interleaved format, especially for small thread block sizes, likely due to better cache utilization. As thread parallelism increases, execution times stabilize across all configurations. The slight visual differences in output images are likely due to interpolation artifacts from upscaling/downscaling and minor information loss during PPM-to-PNG conversion. The results can be seen in images 1.1, 1.2, 1.3, 1.4, 1.5 and 1.6

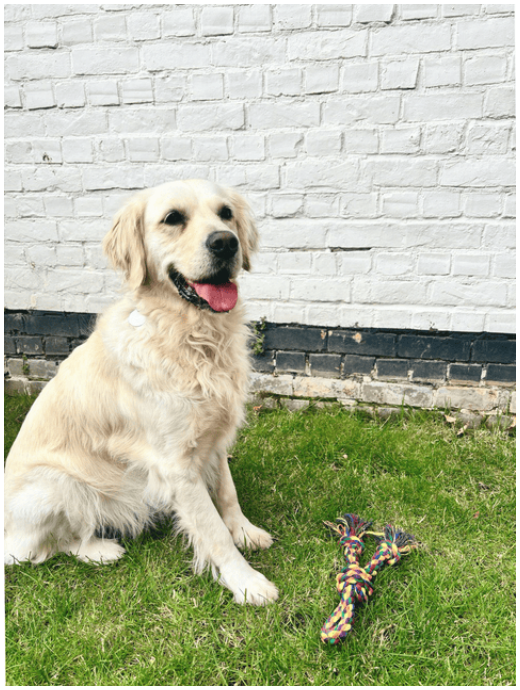
1.1.2 Images



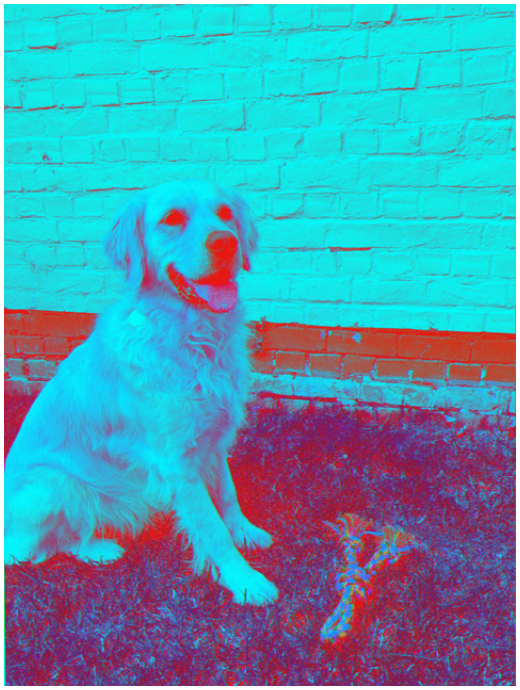
Figuur 1.1: Orginal image



Figuur 1.2: Processed image



Figuur 1.3: Orginal downscaled image



Figuur 1.4: Processed downscaled image



Figuur 1.5: Orginal upscaled image



Figuur 1.6: Processed upscaled image

1.1.3 Guiding questions - Part 1

Question 1: What is the performance difference between the coalesced and uncoalesced kernel?

Coalesced memory access consistently results in lower execution times compared to uncoalesced access across all input sizes and configurations. The difference is most noticeable for smaller thread block sizes, where uncoalesced access introduces higher memory latency due to inefficient global memory transactions. However, as the number of threads per block increases, the performance gap narrows, suggesting that higher parallelism mitigates some of the inefficiencies of uncoalesced access.

Question 2: How does the performance difference change dependent on input and/ or grid size?

The performance difference between coalesced and uncoalesced memory access decreases as the input size grows, but the overall execution time increases. Larger images require more processing, amplifying the impact of inefficient memory access, yet the performance gap remains relatively stable across resolutions. Regarding grid size, smaller thread block sizes (e.g., 1 or 2 threads per block) show a more pronounced difference between coalesced and uncoalesced access due to higher memory latency. As the number of threads per block increases, execution times stabilize, and the difference between coalesced and uncoalesced access diminishes, indicating that higher parallelism mitigates memory access inefficiencies.

1.2 Part 2: Global, shared and constant memory

In this chapter, we analyze the impact of different memory architectures on GPU performance by comparing three approaches:

- Using only global memory
- Utilizing both global and constant memory
- Leveraging global and shared memory

We perform time measurements for matrix multiplication across various input sizes and grid configurations to evaluate their efficiency. Additionally, we measure memory migration times, specifically the transfer from CPU to GPU global memory and from GPU global memory to GPU shared memory. These analyses provide insights into optimizing memory usage for GPU-based computations. The code snippet can be found in attachment 1.2.

1.2.1 Migration time

Matrix Size	CPU to GPU Global Migration Time (ns)	GPU Global to GPU Shared Migration Time (ns)
2 x 2	170,30	566,00
4 x 4	126,88	568,00
8 x 8	231,87	574,00
16 x 16	71,68	594,00
32 x 32	93,12	690,00

Tabel 1.4: Migration times between CPU and GPU memory for different matrix sizes.

In table 1.4 we can see that the times for migration from CPU to GPU global memory and from GPU global memory to GPU shared memory increase with the matrix size, but they do so relatively slowly. However, there is a more significant increase in the migration time from GPU global memory to GPU shared memory as matrix sizes grow. This is somewhat expected because shared memory is physically located closer to the compute units of the GPU, which means that it typically requires a more complex data transfer process when moving larger chunks of data. The noticeable difference in migration times suggests that larger matrices, with their larger data requirements, place more strain on memory transfer and can exacerbate the overhead of migrating data between global and shared memory. This behavior aligns with the idea that the shared memory is designed for small, fast-access data which can be reused, and moving large matrices into shared memory can cause inefficiencies.

1.2.2 Different memory types

In table 1.5, the processing times show that for smaller matrices (2x2 and 4x4), the global and constant memory configuration performs best, as constant memory provides faster

Matrix size	Global only memory [ns]	Global and constant memory [ns]	Global and shared memory [ns]
2 x 2	71,99	62,70	683,87
4 x 4	72,76	62,44	658,04
8 x 8	73,34	55,03	539,33
16 x 16	77,16	84,83	522,55
32 x 32	81,84	89,44	662,22

Tabel 1.5: Execution time for different memory types

access for read-only data. However, as the matrix size increases, the global only memory configuration becomes more efficient, outperforming both global and constant memory, and significantly surpassing global and shared memory, which is the slowest. This suggests that, while shared memory provides benefits in terms of data locality, the overhead of managing shared memory outweighs its advantages for larger matrices, leading to longer processing times compared to global memory.

1.2.3 Guiding questions - Part 2

Question 1: How much time does memory migration take (CPU to GPU global; GPU global to GPU shared)?

The migration times from CPU to GPU global memory and from GPU global to GPU shared memory are measured for different matrix sizes. For migration from CPU to GPU global memory, the times vary between 71.68 ns (for 16x16 matrices) and 170.30 ns (for 2x2 matrices). The migration from GPU global to GPU shared memory shows times ranging from 522.55 ns (for 16x16 matrices) to 690.00 ns (for 32x32 matrices). As matrix sizes increase, the migration time from GPU global to GPU shared memory increases significantly, suggesting a higher overhead when moving larger data to shared memory.

The increased migration time for larger matrices when transferring from GPU global to GPU shared memory is due to the fact that shared memory is located much closer to the processing cores within the GPU, but it is also much smaller in size compared to global memory. As the matrix size increases, more data must be transferred, which can lead to memory contention and additional overhead in managing the transfer to shared memory.

Question 2: How do the execution times of matrix multiplication compare when employing the three different memory types?

For smaller matrices (2x2 and 4x4), global and constant memory performs the fastest due to the optimized access for read-only data. As matrix size increases, the performance of global only memory surpasses both global and constant memory and global and shared memory, making it the most efficient configuration for larger matrices (16x16 and 32x32). In contrast, global and shared memory consistently shows the slowest execution times, especially for larger matrices, due to the overhead of managing shared memory.

Question 3: How do the times scale with input and/ or grid size?

Migration times increase with larger matrix sizes, especially the transfer from GPU global to GPU shared memory. Similarly, execution times generally increase with larger matrices, though the effect is more pronounced with certain memory configurations. For example, while global and constant memory shows favorable times for smaller matrices, the performance gap increases as the matrix size grows, with global only memory outperforming the other configurations for larger matrices. We expect that the grid size also affects performance, with larger grids causing more complex memory management and potentially higher overhead in configurations that utilize shared memory.

CONCLUSION

In this report, we analyzed the performance impact of memory access patterns and different GPU memory architectures on execution times. Our experiments focused on red pixel inversion using coalesced and uncoalesced memory access and on matrix multiplication employing global, shared, and constant memory.

For coalesced versus uncoalesced memory access, our results demonstrate that coalesced memory consistently outperforms uncoalesced access, particularly when thread block sizes are small. The inefficiencies of uncoalesced memory access lead to higher latency due to non-optimal global memory transactions. As the number of threads per block increases, the performance gap between coalesced and uncoalesced access narrows, suggesting that higher parallelism helps mitigate some inefficiencies. Additionally, the planar data layout proved more efficient than the interleaved format, likely due to improved cache utilization. While execution times increased with image size, the relative trends in performance remained consistent across different resolutions.

In our analysis of global, shared, and constant memory, we found that for smaller matrices, leveraging constant memory in combination with global memory provided the best execution times due to the optimized access for read-only data. However, as matrix sizes grew, global-only memory emerged as the most efficient approach, surpassing both global and shared memory. The shared memory configuration, despite its advantage of fast access, introduced higher overhead for large matrices, leading to longer execution times.

Memory migration analysis revealed that transferring data from CPU to GPU global memory incurs relatively minor overhead, but moving data from GPU global to GPU shared memory exhibits a more substantial increase in migration time as matrix sizes grow. This behavior aligns with the expected limitations of shared memory, which is designed for small, fast-access data reuse but struggles when handling large matrix transfers.

Overall, our findings emphasize the importance of optimizing memory access patterns and choosing the appropriate GPU memory configuration depending on workload characteristics. Efficient memory utilization, particularly through coalesced access and selecting the right memory type, can significantly improve execution performance in GPU-accelerated applications.

Listing 1.1: Code snippet for Coalesced Uncoalesced memory acces

```

#include <cuda_runtime.h>
#include <iostream>
#include <stdint.h>
#include <stdio.h>

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


#define CUDA_CHECK(call) do {                                   \
    cudaError_t err = call;                                     \
    if (err != cudaSuccess) {                                    \
        fprintf(stderr, "CUDA error in %s at line %d: %s\n",     \
                __FILE__, __LINE__, cudaGetErrorString(err));   \
        exit(EXIT_FAILURE);                                      \
    }                                                            \
} while (0)


uint8_t* get_image_array(void){
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET,
       imageFile);
    fclose(imageFile);
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array, const char* filename){
    FILE *imageFile;
    imageFile=fopen(filename,"wb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile,"P6\n");
    fprintf(imageFile,"%d %d\n", M, N);
    fprintf(imageFile,"255\n");

    fwrite(image_array, 1, M*N*C, imageFile);

    fclose(imageFile);
}




// Convert an interleaved RGB image to planar format
```

```
void convert_rgb_to_planar(uint8_t* input, uint8_t* output) {
    for (int i = 0; i < M * N; i++) {
        output[i] = input[3 * i];
        output[i + M * N] = input[3 * i + 1];
        output[i + 2 * M * N] = input[3 * i + 2];
    }
}

// Convert a planar image to interleaved RGB format
void convert_planar_to_rgb(uint8_t* input, uint8_t* output) {
    for (int i = 0; i < M * N; i++) {
        output[3 * i] = input[i];
        output[3 * i + 1] = input[i + M * N];
        output[3 * i + 2] = input[i + 2 * M * N];
    }
}









__global__ void invert_red_coalesced_interleaved(const uint8_t *input,
    uint8_t *output, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        output[idx * 3] = 255 - input[idx * 3];     // Invert red
        output[idx * 3 + 1] = input[idx * 3 + 1];   // Keep green
        output[idx * 3 + 2] = input[idx * 3 + 2];   // Keep blue
    }
}

__global__ void invert_red_uncoalesced_interleaved(const uint8_t *input,
    uint8_t *output, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        int red_idx = (idx % (M * N)) * 3; // Direct index for interleaved
            memory (red)
        output[red_idx] = 255 - input[red_idx];   // Invert red
        output[red_idx + 1] = input[red_idx + 1]; // Keep green
        output[red_idx + 2] = input[red_idx + 2]; // Keep blue
    }
}

__global__ void invert_red_coalesced_planar(const uint8_t *input, uint8_t *
    output, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        output[idx] = 255 - input[idx];                     // Invert red
        output[idx + M * N] = input[idx + M * N];           // Keep green
        output[idx + 2 * M * N] = input[idx + 2 * M * N];   // Keep blue
    }
}

__global__ void invert_red_uncoalesced_planar(const uint8_t *input, uint8_t
     *output, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        int red_idx = idx;  // Direct index for planar memory (red)
        output[red_idx] = 255 - input[red_idx];                     //
```

```
            Invert red
        output[red_idx + M * N] = input[red_idx + M * N];           // Keep
            green
        output[red_idx + 2 * M * N] = input[red_idx + 2 * M * N];   // Keep
            blue
    }
}

void run_kernel(uint8_t *h_image, uint8_t *h_output, bool coalesced, bool
    is_planar) {
    int image_size = M * N * C;
    int pixel_count = M * N;

    uint8_t *d_image, *d_output;
    CUDA_CHECK(cudaMalloc(&d_image, image_size));
    CUDA_CHECK(cudaMalloc(&d_output, image_size));
    CUDA_CHECK(cudaMemcpy(d_image, h_image, image_size,
        cudaMemcpyHostToDevice));

    int threads_per_block = 1;
    int blocks_per_grid = (pixel_count + threads_per_block - 1) /
        threads_per_block;


    if (is_planar) {
        if (coalesced) {
            invert_red_coalesced_planar<<<blocks_per_grid,
                threads_per_block>>>(d_image, d_output, pixel_count);
        } else {
            invert_red_uncoalesced_planar<<<blocks_per_grid,
                threads_per_block>>>(d_image, d_output, pixel_count);
        }
    } else {
        if (coalesced) {
            invert_red_coalesced_interleaved<<<blocks_per_grid,
                threads_per_block>>>(d_image, d_output, pixel_count);
        } else {
            invert_red_uncoalesced_interleaved<<<blocks_per_grid,
                threads_per_block>>>(d_image, d_output, pixel_count);
        }
    }

    for (threads_per_block; threads_per_block <= 1024; threads_per_block
        *=2)
    {
        blocks_per_grid = (pixel_count + threads_per_block - 1) /
            threads_per_block;
        float total_time = 0.0f;
        int num_runs = 100;

        for (int run = 0; run < num_runs; run++) {
            cudaEvent_t start, stop;
            cudaEventCreate(&start);
            cudaEventCreate(&stop);

            cudaEventRecord(start);

            if (is_planar) {
                if (coalesced) {
                    invert_red_coalesced_planar<<<blocks_per_grid,
                        threads_per_block>>>(d_image, d_output, pixel_count
```

```
                        );
164                 } else {
165                     invert_red_uncoalesced_planar<<<blocks_per_grid,
                        threads_per_block>>>(d_image, d_output, pixel_count
                        );
166                 }
167             } else {
168                 if (coalesced) {
169                     invert_red_coalesced_interleaved<<<blocks_per_grid,
                        threads_per_block>>>(d_image, d_output, pixel_count
                        );
170                 } else {
171                     invert_red_uncoalesced_interleaved<<<blocks_per_grid,
                        threads_per_block>>>(d_image, d_output, pixel_count
                        );
172                 }
173             }
174
175             cudaEventRecord(stop);
176
177             CUDA_CHECK(cudaMemcpy(h_output, d_output, image_size,
                    cudaMemcpyDeviceToHost));
178
179             cudaEventSynchronize(stop);
180             float milliseconds = 0;
181             cudaEventElapsedTime(&milliseconds, start, stop);
182
183             total_time += milliseconds;
184
185             cudaEventDestroy(start);
186             cudaEventDestroy(stop);
187         }
188
189         // Calculate average time
190         float average_time = total_time / num_runs;
191         printf("number␣of␣threads:␣%d\n", threads_per_block);
192         printf("number␣of␣blocks:␣%d\n", blocks_per_grid);
193         printf("Average␣kernel␣execution␣time␣(%s,␣%s):␣%.3f␣ms\n",
                coalesced ? "Coalesced" : "Uncoalesced", is_planar ? "Planar" :
                 "Interleaved", average_time);
194
195     }
196
197
198
199     cudaFree(d_image);
200     cudaFree(d_output);
201 }
202
203 int main (void) {
204     uint8_t* image_array = get_image_array();
205     uint8_t* coalesced_output_interleaved = (uint8_t*)malloc(M*N*C);
206     uint8_t* uncoalesced_output_interleaved = (uint8_t*)malloc(M*N*C);
207     uint8_t* coalesced_input_planar = (uint8_t*)malloc(M*N*C);
208     uint8_t* uncoalesced_input_planar = (uint8_t*)malloc(M*N*C);
209     uint8_t* coalesced_output_planar = (uint8_t*)malloc(M*N*C);
210     uint8_t* uncoalesced_output_planar = (uint8_t*)malloc(M*N*C);
211     uint8_t* coalesced_output_planar2 = (uint8_t*)malloc(M*N*C);
212     uint8_t* uncoalesced_output_planar2 = (uint8_t*)malloc(M*N*C);
213
214
```

```
215     convert_rgb_to_planar(image_array, coalesced_input_planar);
216     convert_rgb_to_planar(image_array, uncoalesced_input_planar);
217 
218     // Test voor zowel Planar als Interleaved
219     run_kernel(image_array, coalesced_output_interleaved, true, false); //
            Interleaved Coalesced
220     run_kernel(image_array, uncoalesced_output_interleaved, false, false);
            // Interleaved Uncoalesced
221     run_kernel(coalesced_input_planar, coalesced_output_planar, true, true)
            ;  // Planar Coalesced
222     run_kernel(uncoalesced_input_planar, uncoalesced_output_planar, false,
            true); // Planar Uncoalesced
223 
224 
225     convert_planar_to_rgb(coalesced_output_planar, coalesced_output_planar2
            );
226     convert_planar_to_rgb(uncoalesced_output_planar,
            uncoalesced_output_planar2);
227 
228     save_image_array(coalesced_output_interleaved, "
            coalesced_output_interleaved.ppm");
229     save_image_array(uncoalesced_output_interleaved, "
            uncoalesced_output_interleaved.ppm");
230     save_image_array(coalesced_output_planar2, "coalesced_output_planar.ppm
            ");
231     save_image_array(uncoalesced_output_planar2, "uncoalesced_output_planar
            .ppm");
232 
233     free(image_array);
234     free(coalesced_output_interleaved);
235     free(uncoalesced_output_interleaved);
236     free(coalesced_input_planar);
237     free(uncoalesced_input_planar);
238     free(coalesced_output_planar);
239     free(uncoalesced_output_planar);
240     free(coalesced_output_planar2);
241     free(uncoalesced_output_planar2);
242 
243     return 0;
244 }
```

Listing 1.2: Code snippet for different types of memory

```
1
2  #include <stdio.h>
3  #include <cuda.h>
4
5  #define N 32 // Matrix size is N x N
6
7  __constant__ int d_B_fixed[N*N]; // Constant memory for B
8
9  // Kernel with only global memory
10 __global__ void matMulGlobal(int *A, int *B, int *C, int n) { //A,B,C are
       pointers to the matrices, n is the size of the matrices
11     int row = blockIdx.y * blockDim.y + threadIdx.y;
12     int col = blockIdx.x * blockDim.x + threadIdx.x;
13     if (row < n && col < n) {
14         int sum = 0;
15         for (int k = 0; k < n; k++) {
16             sum += A[row * n + k] * B[k * n + col]; //Matrix multiplication
                   for C[row][col]
17         }
18         C[row * n + col] = sum;
19     }
20 }
21
22 // Kernel with shared memory
23 __global__ void matMulShared(int *A, int *B, int *C, int n) {
24     __shared__ int s_A[N][N];
25     __shared__ int s_B[N][N];
26
27     int row = threadIdx.y;
28     int col = threadIdx.x;
29
30
31     clock_t start = clock();
32
33     s_A[row][col] = A[row * n + col];
34     s_B[row][col] = B[row * n + col];
35     __syncthreads();
36
37     clock_t stop = clock();
38     if (row == 0 && col == 0) {
39         printf("Time␣GPU␣global␣to␣GPU␣shared:␣%f\n", (double)(stop - start
               ) / CLOCKS_PER_SEC);
40         printf("\n");
41     }
42
43
44
45     int sum = 0;
46     for (int k = 0; k < n; k++) {
47         sum += s_A[row][k] * s_B[k][col];
48     }
49     C[row * n + col] = sum;
50 }
51
52 // Kernel with constant memory for B
53 __global__ void matMulConstant(int *A, int *C, int n) {
54     int row = blockIdx.y * blockDim.y + threadIdx.y;
55     int col = blockIdx.x * blockDim.x + threadIdx.x;
56     if (row < n && col < n) {
57         int sum = 0;
```

```
58          for (int k = 0; k < n; k++) {
59              sum += A[row * n + k] * d_B_fixed[k * n + col];
60          }
61          C[row * n + col] = sum;
62      }
63  }
64
65  int main() {
66      int h_A[N*N], h_B[N*N], h_C[N*N]; // Host matrices and result matrix
67      int *d_A, *d_B, *d_C;             // Device matrices and result matrix
68
69      // Fill matrices A and B
70      for (int i = 0; i < N*N; i++) {
71          h_A[i] = rand() % 10;
72          h_B[i] = rand() % 10;
73      }
74
75      //print matrix A and B
76      printf("Matrix␣A:\n");
77      for (int i = 0; i < N*N; i++) {
78          printf("%d␣", h_A[i]);
79          if ((i+1) % N == 0) {
80              printf("\n");
81          }
82      }
83      printf("\n");
84
85      printf("Matrix␣B:\n");
86      for (int i = 0; i < N*N; i++) {
87          printf("%d␣", h_B[i]);
88          if ((i+1) % N == 0) {
89              printf("\n");
90          }
91      }
92      printf("\n");
93
94
95
96      cudaMalloc(&d_A, N*N*sizeof(int));
97      cudaMalloc(&d_B, N*N*sizeof(int));
98      cudaMalloc(&d_C, N*N*sizeof(int));
99
100
101
102     //TIME MEASUREMENTS CPU TO GPU GLOBAL
103
104
105
106     cudaEvent_t start_memcpy, stop_memcpy;
107
108     cudaEventCreate(&start_memcpy);
109     cudaEventCreate(&stop_memcpy);
110
111     // Measurement for A and B
112     cudaEventRecord(start_memcpy);
113     cudaMemcpy(d_A, h_A, N*N*sizeof(int), cudaMemcpyHostToDevice);
114     cudaMemcpy(d_B, h_B, N*N*sizeof(int), cudaMemcpyHostToDevice);
115     cudaDeviceSynchronize();
116     cudaEventRecord(stop_memcpy);
117     cudaEventSynchronize(stop_memcpy);
118     float memcpy_time = 0.0f;
```

```
    cudaEventElapsedTime(&memcpy_time, start_memcpy, stop_memcpy);
    printf("Memory migration (CPU to GPU): %f ms\n", memcpy_time);


    cudaEventDestroy(start_memcpy);
    cudaEventDestroy(stop_memcpy);


    cudaMemcpyToSymbol(d_B_fixed, h_B, N*N*sizeof(int), 0,
        cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(N, N); // N x N threads in a block
    dim3 blocksPerGrid(1, 1); // 1 x 1 grid



    matMulGlobal<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
    matMulConstant<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_C, N);
    matMulShared<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);


    float total_time = 0.0f;
    float average_time = 0.0f;
    int num_runs = 100;


    //global only
    for (int run = 0; run < num_runs; run++) {
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        matMulGlobal<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float milliseconds = 0;
        cudaEventElapsedTime(&milliseconds, start, stop);

        total_time += milliseconds;

        cudaMemcpy(h_C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);

        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }

    // Calculate average time
    average_time = total_time / num_runs;
    printf("Global only: Average time: %f ms\n", average_time);

    printf("Matrix C:\n");
    for (int i = 0; i < N*N; i++) {
        printf("%d ", h_C[i]);
        if ((i+1) % N == 0) {
            printf("\n");
        }
    }
```

```
179     printf("\n");
180
181     total_time = 0.0f;
182     average_time = 0.0f;
183
184     //global & constant
185     for (int run = 0; run < num_runs; run++) {
186         cudaEvent_t start, stop;
187         cudaEventCreate(&start);
188         cudaEventCreate(&stop);
189
190         cudaEventRecord(start);
191
192         matMulConstant<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_C, N);
193
194         cudaDeviceSynchronize();
195         cudaEventRecord(stop);
196         cudaEventSynchronize(stop);
197         float milliseconds = 0;
198         cudaEventElapsedTime(&milliseconds, start, stop);
199
200         total_time += milliseconds;
201
202         cudaMemcpy(h_C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
203
204         cudaEventDestroy(start);
205         cudaEventDestroy(stop);
206     }
207
208     // Calculate average time
209     average_time = total_time / num_runs;
210     printf("Global␣and␣constant:␣Average␣time:␣%f␣ms\n", average_time);
211
212
213     printf("Matrix␣C:\n");
214     for (int i = 0; i < N*N; i++) {
215         printf("%d␣", h_C[i]);
216         if ((i+1) % N == 0) {
217             printf("\n");
218         }
219     }
220     printf("\n");
221
222     total_time = 0.0f;
223     average_time = 0.0f;
224
225     //global & shared
226     for (int run = 0; run < num_runs; run++) {
227         cudaEvent_t start, stop;
228         cudaEventCreate(&start);
229         cudaEventCreate(&stop);
230         cudaEventRecord(start);
231
232         matMulShared<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
233
234         cudaDeviceSynchronize();
235         cudaEventRecord(stop);
236         cudaEventSynchronize(stop);
237         float milliseconds = 0;
238         cudaEventElapsedTime(&milliseconds, start, stop);
239
```

```
        total_time += milliseconds;

        cudaMemcpy(h_C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);


        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }

    // Calculate average time
    average_time = total_time / num_runs;
    printf("Global and shared: Average time: %f ms\n", average_time);


    printf("Matrix C:\n");
    for (int i = 0; i < N*N; i++) {
        printf("%d ", h_C[i]);
        if ((i+1) % N == 0) {
            printf("\n");
        }
    }
    printf("\n");

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    return 0;
}
```